Lab 7: Fuzz Testing with Hypothesis

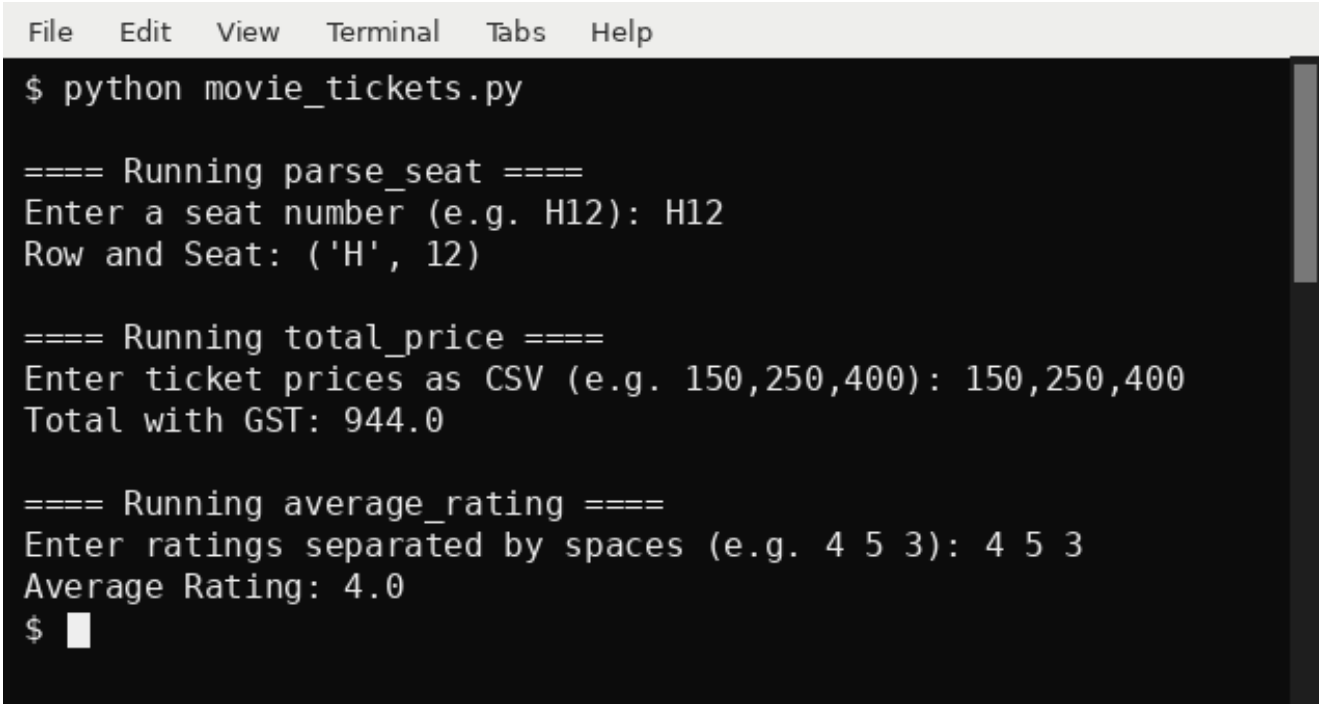
Name: Spoorthi A N | SRN: PES2UG24CS515 | Section: I
Repo: L6_L7_15_movie_tickets (Lab 7 - movie_tickets)

Step 1: Installing dependencies

pip install hypothesis AND pip install pytest

Step 2: Running the buggy application (python movie_tickets.py)

Output of the buggy script with normal inputs:



Step 3: Completed test_movie_tickets.py using Hypothesis

Property-based tests for all three functions: the function must not crash for any text input or None.

Step 4: Running pytest on the buggy code - test cases FAILING

pytest output (part 1 of 2): all 3 tests fail; Hypothesis reports the falsifying examples.



```
File  Edit  View  Terminal  Tabs  Help
$ pytest test_movie_tickets.py

========================================= test session starts ==========================================
platform linux -- Python 3.10.12, pytest-9.2.0, pluggy-1.7.0
rootdir: /tmp/lab7/buggy
plugins: hypothesis-6.168.5
collecting ...
collected 3 items

test_movie_tickets.py FFF                                                                        [100%]

=============================================== FAILURES ===============================================
_____________________________________ test_parse_seat_no_crash _____________________________________
  + Exception Group Traceback (most recent call last):
  |   File "/tmp/lab7/buggy/test_movie_tickets.py", line 5, in test_parse_seat_no_crash
  |     def test_parse_seat_no_crash(s):
  | exceptiongroup.ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "/tmp/lab7/buggy/test_movie_tickets.py", line 7, in test_parse_seat_no_crash
    |     parse_seat(s)
    |   File "/tmp/lab7/buggy/movie_tickets.py", line 2, in parse_seat
    |     return seat[0].upper(), int(seat[1:])
    | ValueError: invalid literal for int() with base 10: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/tmp/lab7/buggy/test_movie_tickets.py", line 9, in test_parse_seat_no_crash
    |     assert False, f"parse_seat crashed with: {e!r}"
    | AssertionError: parse_seat crashed with: ValueError("invalid literal for int() with base 10: ''")
    | assert False
    | Failing test case: test_parse_seat_no_crash(
    |     s='0',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "/tmp/lab7/buggy/test_movie_tickets.py", line 7, in test_parse_seat_no_crash
    |     parse_seat(s)
    |   File "/tmp/lab7/buggy/movie_tickets.py", line 2, in parse_seat
    |     return seat[0].upper(), int(seat[1:])
    | IndexError: string index out of range
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/tmp/lab7/buggy/test_movie_tickets.py", line 9, in test_parse_seat_no_crash
    |     assert False, f"parse_seat crashed with: {e!r}"
    | AssertionError: parse_seat crashed with: IndexError('string index out of range')
    | assert False
    | Failing test case: test_parse_seat_no_crash(
    |     s='',
    | )
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "/tmp/lab7/buggy/test_movie_tickets.py", line 7, in test_parse_seat_no_crash
    |     parse_seat(s)
    |   File "/tmp/lab7/buggy/movie_tickets.py", line 2, in parse_seat
    |     return seat[0].upper(), int(seat[1:])
    | TypeError: 'NoneType' object is not subscriptable
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/tmp/lab7/buggy/test_movie_tickets.py", line 9, in test_parse_seat_no_crash
    |     assert False, f"parse_seat crashed with: {e!r}"
    | AssertionError: parse_seat crashed with: TypeError("'NoneType' object is not subscriptable")
    | assert False
    | Failing test case: test_parse_seat_no_crash(
    |     s=None,
    | )
    +------------------------------------
_____________________________________ test_total_price_no_crash _____________________________________
  + Exception Group Traceback (most recent call last):
  |   File "/tmp/lab7/buggy/test_movie_tickets.py", line 12, in test_total_price_no_crash
  |     def test_total_price_no_crash(s):
  | exceptiongroup.ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "/tmp/lab7/buggy/test_movie_tickets.py", line 14, in test_total_price_no_crash
    |     total_price(s)
    |   File "/tmp/lab7/buggy/movie_tickets.py", line 5, in total_price
    |     prices = [int(p) for p in prices_csv.split(',')]
    |   File "/tmp/lab7/buggy/movie_tickets.py", line 5, in <listcomp>
    |     prices = [int(p) for p in prices_csv.split(',')]
    | ValueError: invalid literal for int() with base 10: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/tmp/lab7/buggy/test_movie_tickets.py", line 16, in test_total_price_no_crash
    |     assert False, f"total_price crashed with: {e!r}"
    | AssertionError: total_price crashed with: ValueError("invalid literal for int() with base 10: ''")
    | assert False
    | Failing test case: test_total_price_no_crash(
```

pytest output (part 2 of 2): ZeroDivisionError, AttributeError, and the FAILED summary.

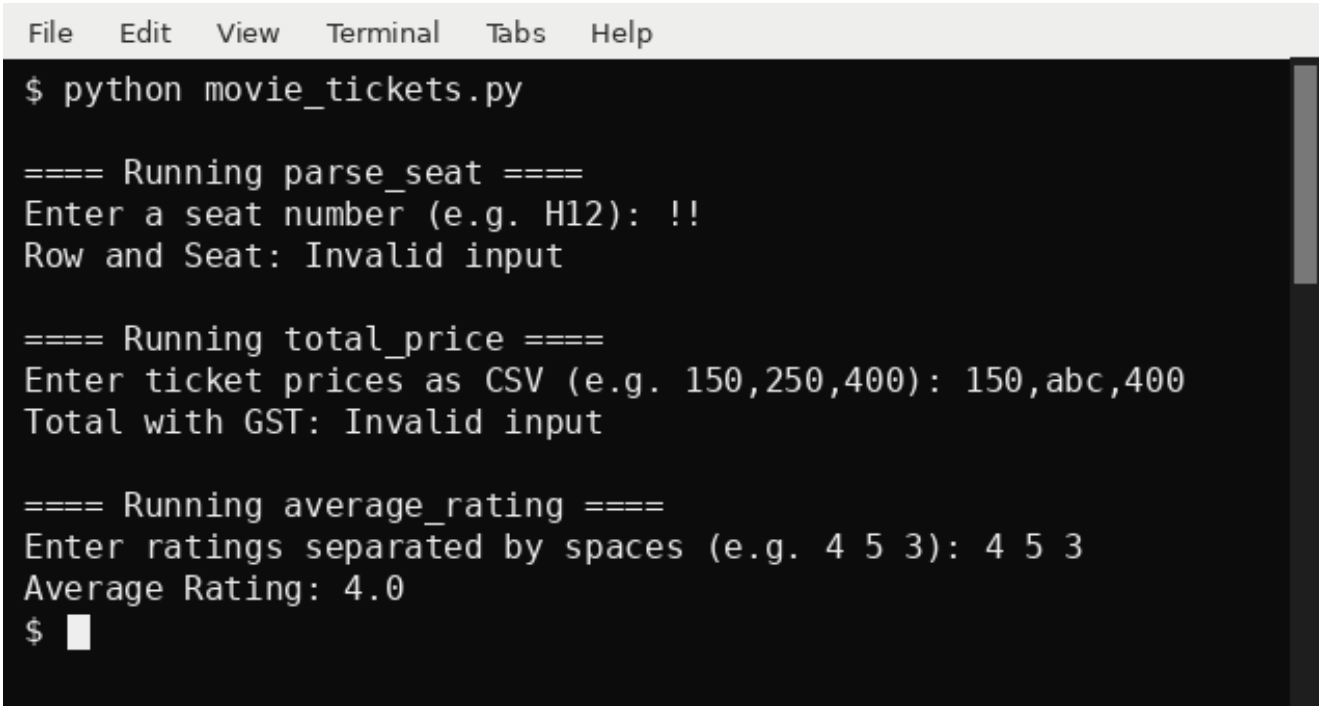


Bugs found by fuzzing: parse_seat crashes with ValueError ('0'), IndexError (") and TypeError (None); total_price crashes with ValueError (") and AttributeError (None); average_rating crashes with ValueError (':'), ZeroDivisionError (") and AttributeError (None).

Step 5: Fixed application (python movie_tickets.py)

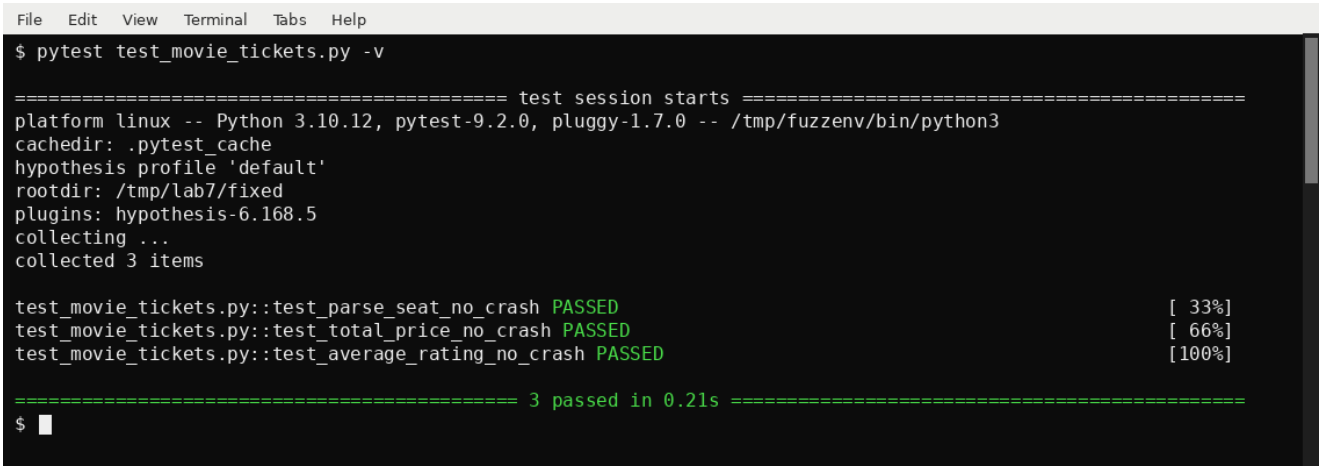
Each function is hardened with try/except around parsing and returns 'Invalid input' instead of crashing on unexpected input.

Fixed script handling invalid inputs without crashing:



Step 6: Re-running pytest against the fixed code - all tests PASS

pytest output: 3 passed.



Appendix A: Fixed movie_tickets.py

```
def parse_seat(seat):
    try:
        row = seat[0].upper()
        number = int(seat[1:])
    except (AttributeError, TypeError, IndexError, ValueError, OverflowError):
        return None
    return row, number

def total_price(prices_csv):
    try:
        prices = [int(p) for p in prices_csv.split(',')]
        total = round(sum(prices) * 1.18, 2)
    except (AttributeError, TypeError, ValueError, OverflowError):
        return None
    return total

def average_rating(ratings):
    try:
        values = [int(r) for r in ratings.split()]
        average = round(sum(values) / len(values), 1)
    except (AttributeError, TypeError, ValueError, ZeroDivisionError, OverflowError):
        return None
    return average

def main():
    print("==== Running parse_seat ====")
    user_input = input("Enter a seat number (e.g. H12): ")
    result = parse_seat(user_input)
    print("Row and Seat:", result if result is not None else "Invalid input")

    print("\n==== Running total_price ====")
    user_input = input("Enter ticket prices as CSV (e.g. 150,250,400): ")
    result = total_price(user_input)
    print("Total with GST:", result if result is not None else "Invalid input")

    print("\n==== Running average_rating ====")
    user_input = input("Enter ratings separated by spaces (e.g. 4 5 3): ")
    result = average_rating(user_input)
    print("Average Rating:", result if result is not None else "Invalid input")

if __name__ == "__main__":
    main()
```

Appendix B: Updated test_movie_tickets.py

```
from hypothesis import given, strategies as st
from movie_tickets import parse_seat, total_price, average_rating

@given(st.text() | st.none())
def test_parse_seat_no_crash(s):
    try:
        parse_seat(s)
    except Exception as e:
        assert False, f"parse_seat crashed with: {e!r}"

@given(st.text() | st.none())
def test_total_price_no_crash(s):
    try:
        total_price(s)
    except Exception as e:
        assert False, f"total_price crashed with: {e!r}"

@given(st.text() | st.none())
def test_average_rating_no_crash(s):
    try:
        average_rating(s)
    except Exception as e:
        assert False, f"average_rating crashed with: {e!r}"
```